Структуры данных:

Связный список

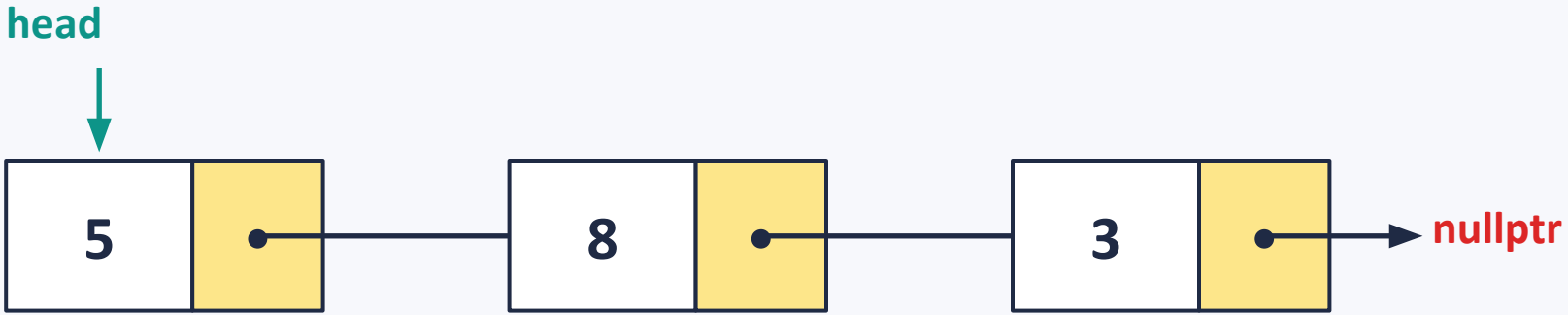
Разминка

Нужно вставить число в начало массива из миллиона элементов.

Что придётся сделать компьютеру?

Подумайте 30 секунд, потом обсудим.

Идея: узлы и ссылки



Узлы лежат в памяти где угодно. Порядок задают не места, а указатели.
Как цепочка людей за руки: чтобы вставить человека, достаточно двоим перехватить руки.

Узел списка

```
struct Node {
    int data;      // значение
    Node* next;    // кто следующий (nullptr — никого)
};

Node* head = nullptr;    // пустой список
```

Единственная точка входа — head. Потеряешь head — потеряешь весь список.

Список против массива

Плюсы

- Вставка и удаление за O(1), если уже знаем место
- Растёт по одному узлу, ничего не сдвигаем

Минусы

- Доступ к k-му элементу: O(n)
- Память под указатели
- На практике vector чаще быстрее: данные лежат рядом, кэш процессора счастлив

Вставка в начало

```
Node* n = new Node{5, head};  // next = старая голова
head = n;                     // n — новая голова
```

Работает и для пустого списка (next станет nullptr), и для непустого (next — старая голова).

Обход списка

```
for (Node* cur = head; cur != nullptr; cur = cur->next) {
    std::cout << cur->data << " ";
}
```

Шагаем по next, пока не уперлись в nullptr. Это O(n).

Удаление первого узла

```
if (head != nullptr) {
    Node* tmp = head;
    head = head->next;
    delete tmp;
}
```

Порядок важен: сначала сдвинуть head, потом удалять. Иначе потеряем остаток списка.

А если удалить из середины?

А если удалить из середины?

Чтобы вырезать узел, нужен его предыдущий сосед.

А если удалить из середины?

Чтобы вырезать узел, нужен его предыдущий сосед.

Назад дороги нет, значит придётся искать его с головы: O(n).

А если удалить из середины?

Чтобы вырезать узел, нужен его предыдущий сосед.

Назад дороги нет, значит придётся искать его с головы: O(n).

Решение: пусть каждый узел знает и предыдущего.

Двусвязный список

```
struct DNode {
    int data;
    DNode* prev;   // у первого — nullptr
    DNode* next;   // у последнего — nullptr
};

DNode* head = nullptr;
DNode* tail = nullptr;
```

Ходим в обе стороны. Цена: еще ?? байт на узел и больше указателей, за которыми надо следить.

Двусвязный список

```
struct DNode {
    int data;
    DNode* prev;   // у первого — nullptr
    DNode* next;   // у последнего — nullptr
};

DNode* head = nullptr;
DNode* tail = nullptr;
```

Ходим в обе стороны. Цена: еще 8 байт на узел и больше указателей, за которыми надо следить.

Двусвязный: вставка в начало

```
DNode* n = new DNode{5, nullptr, head};
if (head != nullptr) head->prev = n;   // список не пуст
else tail = n;                         // пуст: n и голова, и хвост
head = n;
```

Два случая: список пуст или нет. Порядок строк важен, пока head ещё старая.

Двусвязный: удаление по указателю

```
if (p->prev) p->prev->next = p->next; else head = p->next;
if (p->next) p->next->prev = p->prev; else tail = p->prev;
delete p;
```

Знаем узел p — удаляем за O(1), никого не ищем. Это главный выигрыш prev.

Кольцевой список

```
tail->next = head;          // замкнули кольцо

Node* cur = head;           // список не пуст!
do {
    std::cout << cur->data << " ";
    cur = cur->next;
} while (cur != head);      // с while (cur != nullptr) завис бы навсегда
```

Пустой список и список из одного узла (n->next = n) — отдельные случаи, не забывай.

Сравним вместе

	Односвязный	Двусвязный	Кольцевой
Указатели в узле	?	?	?
Ход назад	?	?	?
Удаление по указателю	?	?	?
Лишняя память	?	?	?
Пример	?	?	?

Готовое: std::list

```cpp
std::list<int> lst = {1, 2, 3, 4};
for (int x : lst) std::cout << x << " ";

auto it = lst.begin();    // 1
++it;                     // 2
--it;                     // 1
std::advance(it, 2);      // 3
auto nx = std::next(it);  // 4, it не сдвинулся
auto pv = std::prev(it);  // 2, it не сдвинулся
```

std::list — двусвязный список. advance(it, k) тоже шагает по одному: O(k).

Итог

- Список = узлы + указатели.
- Вставка и удаление дёшево, доступ по индексу дорого.
- prev даёт ход назад, цикл даёт круговой обход.

Задачи

206. Reverse Linked List

разминка, Easy

21. Merge Two Sorted Lists

Easy

2181. Merge Nodes in Between Zeros

Medium

146. LRU Cache*

*: Самостоятельное изучение

Методы std::list: en.cppreference.com/w/cpp/container/list